

# Fixpoint Algorithms for Dataflow Analysis

Yi Wang

# Roadmap of Presentation

1. Motivation + Background
2. Initial fixpoint algorithm: Kildall (1973)
3. Generalization: Cousot, Cousot (1977)
4. Examines efficiency: Hecht, Ullman (1975)
  - a. Worklist algorithm: Kam, Ullman (1976), Cooper, Harvey, Kennedy (2002)
  - b. SCC decomposition: Tarjan (1972)

# Motivation: Static Analysis

Given a program, we want to extract information about the program that's true for all executions, without running it

Why study this:

- Safety (e.g. detect possible division by zero errors)
- Optimizations (e.g. substitute constants for variables)

```
int x;  
scanf("%d", &x);  
int y = 10 / x;
```

```
int x;  
if (input > 0) {  
    x = 5;  
} else {  
    x = 5;  
}  
int y = x + 2;
```

# Solution: Dataflow Analysis on Control-Flow Graphs

Kildall (1973), “A Unified Approach to Global Program Optimization”

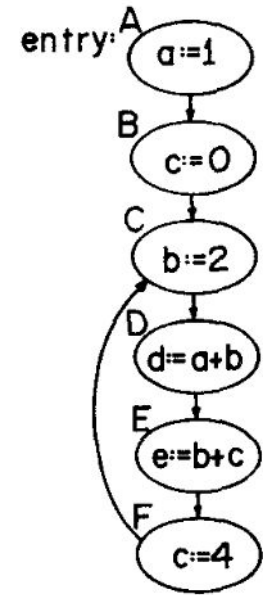
- Before, static analyses were mostly ad-hoc
- Defines general approach: run fixpoint algorithm on control-flow graphs
- Looks at special case of constant propagation
  - Determine which variables can be treated as constants
- Provides justification and describes implementation

# Background: Control-Flow Graph (CFG)

For every program, can define such a directed graph:

- Nodes correspond to statements in the program
- Edges represent possible control flow during execution

```
begin integer i,a,b,c,d,e;  
a:=1; c:=0; . . .  
for i:=1 step 1 until 10 do  
  begin b:=2; . . .  
  d:=a+b; . . .  
  e:=b+c; . . .  
  c:=4; . . .  
  end  
end
```

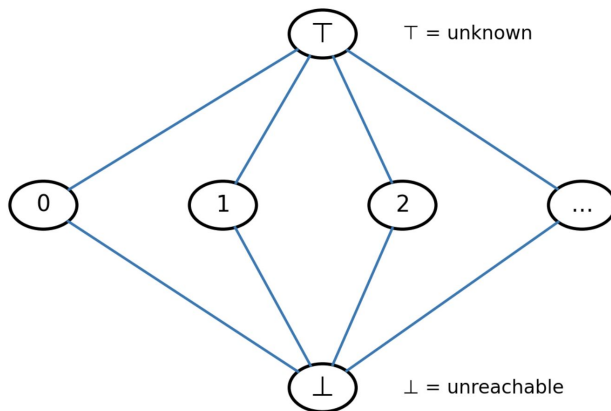


# Background: Lattices

Lattice: partially ordered set  $(L, \leq)$ , so that for all  $x, y \in L$  there exists:

- Least upper bound  $x \sqcup y$ , greatest lower bound  $x \sqcap y$
- Example:  $1 \sqcup 4 = \top$ ,  $1 \sqcap \perp = 1$

Constant Propagation Lattice



# Fixpoint Algorithm Overview

At each node, use elements in  $L$  to represent the set of possible values a variable can have

Initialize all nodes have all variables =  $\perp$

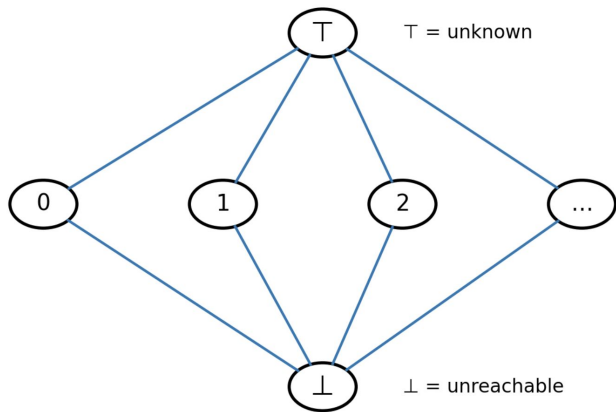
## Update step for a node:

- Join all predecessor states
- Apply statement to obtain new state

## Repeatedly update until no change

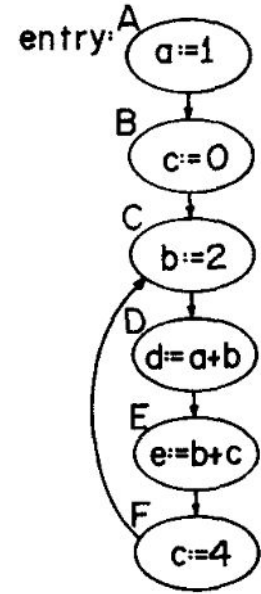
- Fixpoint reached, return all states

## Constant Propagation Lattice



## Example Run: Iteration 0 (initialization)

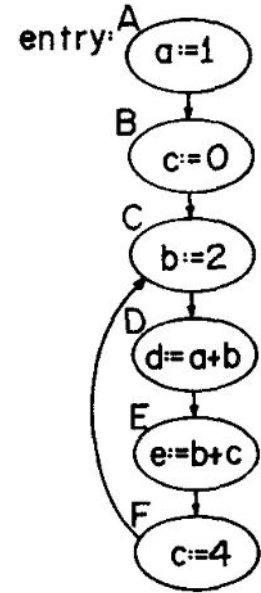
- A:  $\{a = \perp, c = \perp, b = \perp, d = \perp, e = \perp\}$
- B:  $\{a = \perp, c = \perp, b = \perp, d = \perp, e = \perp\}$
- C:  $\{a = \perp, c = \perp, b = \perp, d = \perp, e = \perp\}$
- D:  $\{a = \perp, c = \perp, b = \perp, d = \perp, e = \perp\}$
- E:  $\{a = \perp, c = \perp, b = \perp, d = \perp, e = \perp\}$
- F:  $\{a = \perp, c = \perp, b = \perp, d = \perp, e = \perp\}$





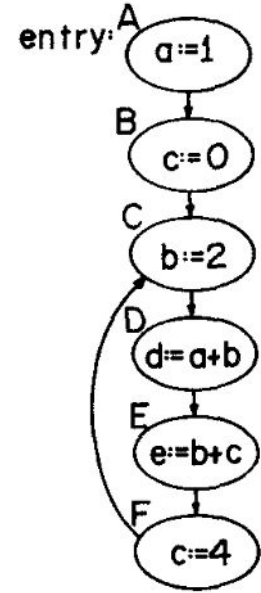
# Example Run: Iteration 1

- A:  $\{a = 1, c = \perp, b = \perp, d = \perp, e = \perp\}$
- B:  $\{a = 1, c = 0, b = \perp, d = \perp, e = \perp\}$
- C:  $\{a = 1, c = 0, b = 2, d = 3, e = \perp\}$
- D:  $\{a = 1, c = 0, b = 2, d = 3, e = \perp\}$
- E:  $\{a = 1, c = 0, b = 2, d = 3, e = 2\}$
- F:  $\{a = 1, c = 4, b = 2, d = 3, e = 2\}$



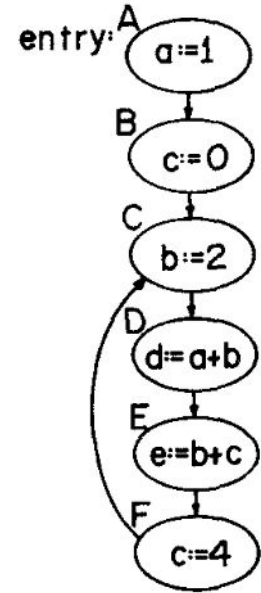
## Example Run: Iteration 2

- A:  $\{a = 1, c = \perp, b = \perp, d = \perp, e = \perp\}$
- B:  $\{a = 1, c = 0, b = \perp, d = \perp, e = \perp\}$
- C:  $\{a = 1, c = \top, b = 2, d = 3, e = 2\}$
- D:  $\{a = 1, c = \top, b = 2, d = 3, e = 2\}$
- E:  $\{a = 1, c = \top, b = 2, d = 3, e = \top\}$
- F:  $\{a = 1, c = 4, b = 2, d = 3, e = \top\}$



## Example Run: Iteration 3

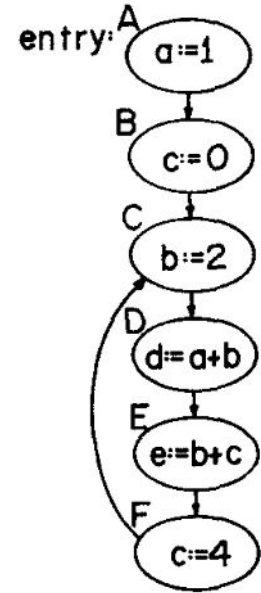
- A:  $\{a = 1, c = \perp, b = \perp, d = \perp, e = \perp\}$
- B:  $\{a = 1, c = 0, b = \perp, d = \perp, e = \perp\}$
- C:  $\{a = 1, c = \top, b = 2, d = 3, e = \top\}$
- D:  $\{a = 1, c = \top, b = 2, d = 3, e = \top\}$
- E:  $\{a = 1, c = \top, b = 2, d = 3, e = \top\}$
- F:  $\{a = 1, c = 4, b = 2, d = 3, e = \top\}$



## Example Run: Iteration 4

- A:  $\{a = 1, c = \perp, b = \perp, d = \perp, e = \perp\}$
- B:  $\{a = 1, c = 0, b = \perp, d = \perp, e = \perp\}$
- C:  $\{a = 1, c = \top, b = 2, d = 3, e = \top\}$
- D:  $\{a = 1, c = \top, b = 2, d = 3, e = \top\}$
- E:  $\{a = 1, c = \top, b = 2, d = 3, e = \top\}$
- F:  $\{a = 1, c = 4, b = 2, d = 3, e = \top\}$

Check: fixpoint reached



# Limitations of Kildall (1973)

Only looks at constant propagation

Fixpoint algorithm uses naive round-robin propagation

- Looks at every node in each iteration
- Efficiency could be improved

# Abstract Interpretation

Cousot, Cousot (1977), “Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints”

Instead of exact constants, we can have any abstract domain

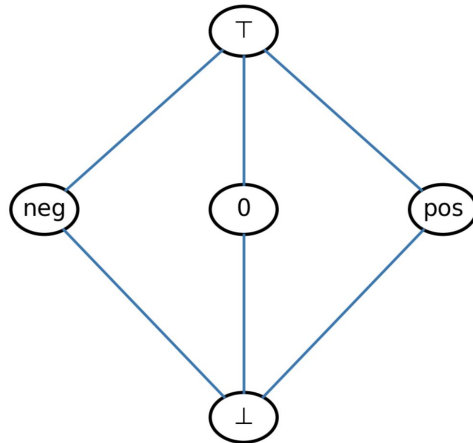
- Example: signs {0, positive, negative}
- Regardless of the domain, we can use fixpoint algorithm

# Abstract Interpretation, continued

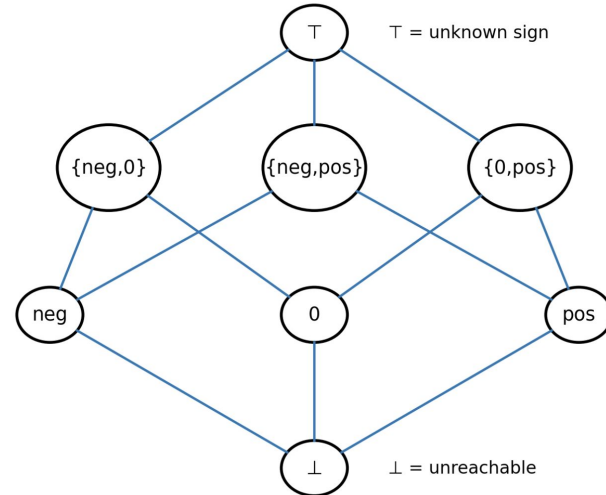
Uses general lattices

- Proves that if lattices have finite height, algorithm is guaranteed to terminate
- Tradeoff between efficiency and precision

Sign Analysis Lattice



Refined Sign Analysis Lattice



# Analyzing Efficiency

Hecht, Ullman (1975) “A simple algorithm for global data flow analysis problems”

Examines how runtime of fixpoint algorithm depends on lattice height  $H$

- No explicit complexity analysis

Naive round-robin: repeatedly scan nodes that receive no updates

Worst case runtime of round-robin:  $O(H * |\text{Var}| * |V| * |E|)$



# Worklist Algorithm

Kam, Ullman (1976), “Global Data Flow Analysis and Iterative Algorithms”

Idea: recompute node only when one of its predecessors changed

- Maintains “worklist” of nodes that need to be updated
- Every time a node’s state is changed, add all successors to worklist

Only implicitly mentions the algorithm

Improved runtime:  $O(H * |Var| * |E|)$

# Worklist Algorithm, continued

Cooper, Harvey, Kennedy (2002), “Iterative Dataflow Analysis, Revisited”

Examines different implementations of the worklist algorithm

- Worklist implemented with different data structures (stack, queue, etc.)
- Evaluated performance using various experiments
  - Potentially speeds up by 40%
- Ordering matters; choice of orders can drastically improve performance
  - No clear best choice

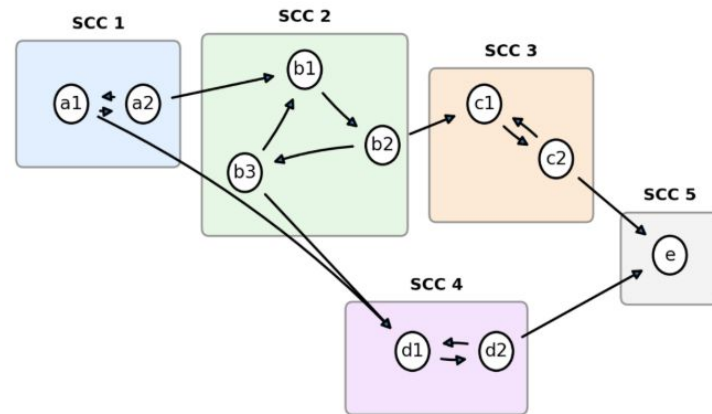
# Strongly Connected Component (SCC) Decomposition

Tarjan (1972), “Depth-first search and linear graph algorithms”

SCC: subgraph for which any node can reach any other node (e.g. cycles)

SCC Decomposition: partitions vertices into SCCs

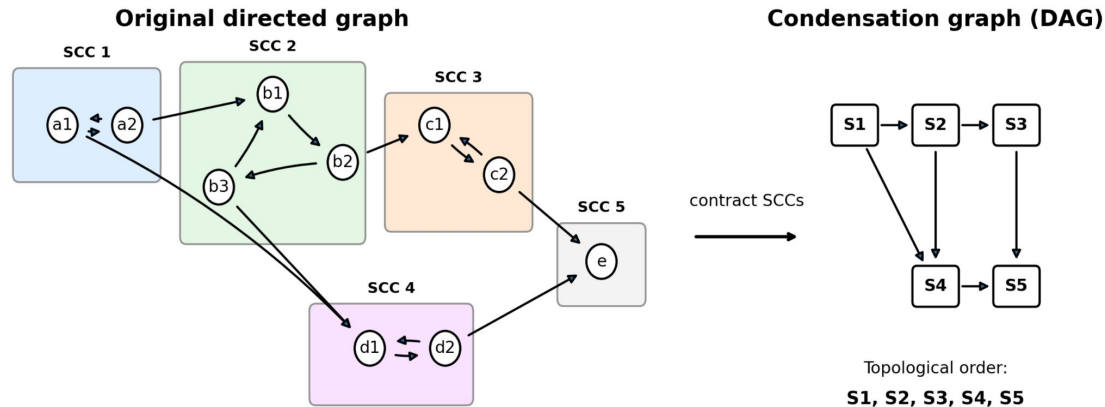
- Can be computed in  $O(|E|)$  time



# Topological Order in Directed Acyclic Graphs (DAGs)

Treating each SCC as a single node, the graph becomes a DAG

DAGs have topological order: can order vertices so that edges only go forward in the ordering



# Applying SCC Decomposition to Compute Fixpoint

Idea: look at each SCC in the CFG individually

- Local computations; more efficient
- Use worklist/naive round robin locally

Process SCCs in topological order of the induced DAG

- SCCs earlier in the order does not depend on later SCCs

# Open problems

## Improving ordering strategy

- Possible to choose best worklist ordering?
- Possible to predict which node stabilize faster?

## Understand precision vs efficiency tradeoff

- How to choose the “right” abstraction for each program?

Thank you!